
# TRANSACTRITE

## SRS – SOFTWARE REQUIREMENT SPECIFICATION

BY:

9506/22 Avneet Kaur

9509/22 Gauri Sharma

9511/22 Ishmeet Kaur

TO:

Dr. Santosh Kumar Yadav

LECTURER

CSE Department

# CONTENT

# Login Screen Module

## 1. Introduction

### *1.1 Purpose*

The purpose is to define the **Software Requirements Specification (SRS)** for the **Login Screen Module** of TransactRite application. This module ensures secure user authentication using **email/password login**, **fingerprint authentication**, and **Firebase Authentication**.

### *1.2 Scope*

The **Login Screen Module** is a critical component of the banking application, ensuring secure access to user accounts. It will include:

- **Email and password authentication** using Firebase
- **Fingerprint authentication (biometric login)** for quick access
- **Secure storage of login sessions** to maintain authentication state
- **Forgot Password feature** for password recovery
- **Two-factor authentication (2FA) support** (optional for additional security)
- **Material Design UI** for an intuitive user experience

The login module enhances security, ensuring that only verified users can access banking features.

### *1.3 Intended Audience and Reading Suggestions*

This document is intended for:

- **Developers** – To implement the login module
- **Testers** – To validate authentication features
- **Project Managers** – To oversee development and integration
- **Stakeholders** – To understand security mechanisms in the banking app

### *1.4 References*

- Firebase Authentication Documentation
- Android Biometric Authentication Guide
- Material Design Guidelines

---

## 2. Overall Description

### *2.1 Product Perspective*

The **Login Screen Module** is an essential security layer of the banking app, integrating with:

- **Firebase Authentication** for user identity management
- **Biometric Authentication API** for fingerprint-based login
- **Room Database (if needed)** for local session storage
- **Encrypted SharedPreferences** for securely storing authentication tokens

## *2.2 Product Functions*

The login module will include:

- **User Registration** – New users can sign up with email and password
- **User Login** – Existing users can log in with credentials
- **Fingerprint Login** – Users can authenticate using biometrics
- **Forgot Password** – Allows users to reset their password via email
- **Session Management** – Keeps users logged in securely
- **Logout Functionality** – Allows users to sign out securely
- **Multi-Factor Authentication (MFA) (Optional)** – Enhances security for high-value transactions

## *2.3 User Characteristics*

- **Bank Customers** – Users with an account in the banking system
- **Security-Conscious Users** – Users who prefer biometric authentication for security
- **General Mobile Users** – Users with basic knowledge of mobile banking apps

---

# 3. Specific Requirements

## *3.1 Functional Requirements*

| ID | Requirement | Description |
|---|---|---|
| FR-1 | User Registration | Users can register using email & password via Firebase |
| FR-2 | Email/Password Login | Users can log in using Firebase Authentication |
| FR-3 | Fingerprint Login | Users can log in using biometric authentication |
| FR-4 | Forgot Password | Users can reset their password via email verification |
| FR-5 | Session Management | Users remain logged in unless they manually log out |
| FR-6 | Logout | Users can securely log out of their account |
| FR-7 | Multi-Factor Authentication (MFA) | Optional feature for enhanced security |

## *3.2 Non-Functional Requirements*

| ID | Requirement | Description |
|---|---|---|
| NFR-1 | Security | User credentials must be securely stored using Firebase Authentication and encrypted storage |
| NFR-2 | Performance | Login authentication should take no more than 2 seconds |

| | | |
|---|---|---|
| NFR-3 | Usability | The UI must follow Material Design principles for a seamless user experience |
| NFR-4 | Compatibility | The module must work on Android 8.0 (Oreo) and above |

---

# 4. External Interface Requirements

## *4.1 User Interfaces*

- **Login Screen:** Fields for email/password, fingerprint login button, and "Forgot Password" link
- **Registration Screen:** Fields for email, password, and confirm password
- **Error Messages:** Proper validation messages for incorrect inputs

## *4.2 Software Interfaces*

- **Firebase Authentication** for user login and registration
- **Android Biometric API** for fingerprint authentication
- **Room Database (if needed)** for local storage of session data

## *4.3 Hardware Interfaces*

- **Fingerprint Sensor** (Required for biometric authentication)

---

# 5. Constraints

- The app must have an active internet connection for Firebase Authentication.
- Fingerprint login requires a device with a fingerprint sensor.
- The app must follow banking security compliance guidelines.

---

# 6. Assumptions and Dependencies

- Users have a registered banking account.
- The device supports **Google Play Services** for Firebase Authentication.
- Biometric authentication is enabled in the device settings.

# DASHBOARD SCREEN & BUTTON FUNCTIONALITY

---

## 1. Introduction

### *1.1 Purpose*

The Dashboard Screen is a core component of the Banking App, providing users with quick access to key financial services. This module ensures an intuitive and responsive interface with seamless navigation to essential banking functionalities, including account management, transactions, bill payments, and user notifications.

### *1.2 Scope*

This module will:

- Display account summary and balance information.
- Provide quick access to key banking features via buttons.
- Ensure smooth user interactions with responsive UI elements.
- Implement security measures for safe financial transactions.
- Log user interactions for enhanced tracking and analytics.

### *1.3 Overview*

This document outlines the functional and non-functional requirements, software dependencies, and constraints for the Dashboard Screen and its button functionalities.

---

## 2. Functional Requirements

### *2.1 Dashboard UI Design*

- The app shall display the user's account summary, including balance and transaction history.
- The UI shall contain quick-access buttons for essential banking features.
- The dashboard shall have a notification section for alerts and updates.

### *2.2 Button Functionalities*

- **Money Transfer**: Opens the fund transfer module with input fields for recipient details and amount.
- **Bill Payments**: Provides access to utilities, mobile recharge, and other payment services.
- **Transaction History**: Displays a categorized view of past transactions.
- **Credit Score Check**: Opens the credit score module for analysis and report generation.
- **Fake Note Detection**: Launches the module for detecting counterfeit currency using image processing.
- **Voice Control**: Enables voice-based navigation and transaction execution for accessibility.

### *2.3 User Feedback & Logging*

- The app shall provide instant feedback on successful or failed actions.
- All dashboard interactions shall be logged securely for tracking purposes.
- Users shall receive error messages in case of connectivity or processing issues.

---

## 3. Non-Functional Requirements

### *3.1 Performance*

- The dashboard shall load within 2 seconds.
- Button responses shall execute actions within 1 second.

### *3.2 Compatibility*

- The module shall be compatible with Android 8.0 (API 26) and above.
- It must function on different screen sizes and orientations.

---

## 4. Software & Hardware Requirements

### *4.1 Software Requirements*

- Android Studio Ladybug 2024.2.2
- Kotlin DSL for build configurations
- Gradle 8.8 or higher
- Firebase for authentication and transaction logging

### *4.2 Hardware Requirements*

- **Minimum Device Specs:**
  - RAM: 2GB+
  - Screen: 5.5"+
  - CPU: ARM 64-bit architecture
  - Storage: 500MB available

---

## 5. Constraints & Limitations

- Real-time transactions depend on internet connectivity.
- Processing speed may vary based on device hardware.
- The first version supports Indian banking services only.

---

## 6. Future Enhancements

- AI-powered financial recommendations.
- Integration with UPI and international payments.
- Cloud-based fraud detection system.

---

## 7. References


- Android Material Design Guidelines
- Firebase Authentication & Realtime Database

# Fund Transfer Module

## 1. Introduction

The Fund Transfer and Transaction History Module is an integral component of a mobile banking or financial application that enables users to transfer money between accounts and view their past transactions. Built using Android Studio with Java and Firebase as the backend, this module provides a seamless and secure financial transaction experience.

## 2. Scope

This module facilitates peer-to-peer (P2P) transfers, fund transfers between accounts, and maintains a history of all transactions in Firebase. It can be integrated into various financial applications such as digital wallets, banking apps, and e-commerce platforms. The scope of this module includes:

- Secure fund transfers between users.
- Real-time update of transaction history.
- Authentication and authorization for secure transactions.
- Firebase as a cloud-based real-time database for transaction storage.

## 3. Purpose

The purpose of this module is to:

- Provide a secure and reliable way for users to transfer funds.
- Maintain a real-time transaction history for tracking expenses.
- Ensure data integrity and security using Firebase Authentication and Firestore.
- Enable easy integration with third-party APIs and future payment gateways.

## 4. Overview

This module consists of two key functionalities:

1. **Fund Transfer**: Users can transfer funds between accounts after authentication and authorization.
2. **Transaction History**: Users can view details of past transactions, including date, time, amount, and recipient details.

The module follows the **Model-View-Controller (MVC)** architecture for efficient code management and scalability.

## 5. Functional Requirements

### 5.1 *Fund Transfer Functionalities*

- User authentication via Firebase Authentication (Email/Password, Google Sign-In, etc.).
- Selection of recipient from the Firebase database.
- Input of transfer amount and transaction verification.
- Real-time update of sender and recipient balances in Firestore.
- Confirmation and success/failure notifications.

### 5.2 *Transaction History Functionalities*

- Fetching and displaying transaction history from Firebase Firestore.
- Filtering transactions based on date, amount, and recipient.
- Real-time synchronization of new transactions.
- Visual representation using charts (optional for spending analytics).

---

## 6. Non-Functional Requirements

- **Security**: All transactions should be encrypted using SSL/TLS.
- **Performance**: The app should fetch and update transactions in real-time with minimal latency.
- **Scalability**: The module should support a growing number of users without performance degradation.
- **Usability**: The user interface should be intuitive and easy to navigate.

---

## 7. Hardware and Software Requirements

### 7.1 *Hardware Requirements*

- Android smartphone (API Level 21 and above)
- Minimum 2GB RAM, Recommended 4GB+
- Internet connection for Firebase access

### 7.2 *Software Requirements*

- Android Studio (Latest Version)
- Java (JDK 8 or above)
- Firebase Console (for Firestore, Authentication, and Cloud Functions)
- Google Play Services

---

## 8. Constraints and Limitations

- Dependent on internet connectivity for Firebase operations.
- Requires Google account authentication for Firebase integration.
- Firebase free-tier has limitations on read/write operations per second.
- No built-in support for real-time bank payments (limited to app-based transactions).

---

## 9. Future Enhancements

### 9.1 *Integration with Real-Time Payment Gateway*

- Implementing a payment gateway like Razorpay, PayPal, Stripe, or UPI.
- Support for direct bank transfers through UPI, NEFT, or IMPS.
- Multi-currency support for international transactions.
- AI-based spending analytics and fraud detection.

### 9.2 *Improved UI/UX Features*

- Dark mode and customization.
- Push notifications for transaction updates.
- QR Code-based fund transfers.

With these future enhancements, the module can be extended to a full-fledged digital payment system.

# **FAKE NOTE DETECTION (**Image Processing Module (OpenCV))

---

## 1. Introduction

### *1.1 Purpose*

The Image Processing Module is a crucial part of the Fake Note Detection module of TransactRite android finance system. It leverages OpenCV to process images captured from the device's camera, analyze currency notes, and classify them as **genuine** or **counterfeit**.

### *1.2 Scope*

This module will:

- Capture images using CameraX.
- Preprocess images (grayscale conversion, edge detection, noise removal).
- Extract key features from currency notes using OpenCV techniques.
- Use machine learning models (TensorFlow Lite) to classify currency notes.
- Display results and provide user feedback.

### *1.3 Overview*

This document outlines the functional and non-functional requirements, software dependencies, and constraints for the Image Processing Module.

---

## 2. Functional Requirements

### *2.1 Image Acquisition*

- The app shall use the device camera to capture currency note images.
- The image capture process shall use **CameraX** for better control over focus and exposure.

### *2.2 Image Preprocessing*

- Convert captured images to **grayscale**.
- Apply **Gaussian Blur** to reduce noise.
- Perform **Edge Detection (Canny)** to highlight currency features.
- Extract **Regions of Interest (ROI)** such as watermarks, security threads, and serial numbers.

### *2.3 Feature Extraction*

- Extract **Key Points** using ORB (Oriented FAST and Rotated BRIEF) or SIFT.

- Perform **Histogram Analysis** to differentiate genuine and fake notes.
- Use **Contour Detection** to analyze note patterns.

### *2.4 Fake Note Classification*

- The app shall integrate with **TensorFlow Lite (TFLite)** for real-time classification.
- Based on extracted features, classify the note as **genuine** or **fake**.
- Display the classification result on the UI.

### *2.5 User Feedback & Logging*

- If a fake note is detected, the app shall alert the user.
- The app shall store scan logs in local storage for future reference.

---

## 3. Non-Functional Requirements

### *3.1 Performance*

- Image processing should take less than **2 seconds**.
- The module shall work on devices with **2GB RAM and above**.

### *3.2 Compatibility*

- The module shall be compatible with **Android 8.0 (API 26) and above**.
- It must work on various Android screen sizes and orientations.

### *3.3 Security & Privacy*

- Captured images shall not be stored permanently.
- User data shall not be shared without consent.

### *3.4 Scalability*

- The module should allow future integration of advanced AI models.
- It should support multi-currency detection in future versions.

---

## 4. Software & Hardware Requirements

### *4.1 Software Requirements*

- **Android Studio Ladybug 2024.2.2**
- **Kotlin DSL for build configurations**
- **Gradle 8.8 or higher**

- **OpenCV 4.7.0 for Android**
- **TensorFlow Lite for ML integration**

### *4.2 Hardware Requirements*

- **Minimum Device Specs:**
  - RAM: **2GB+**
  - Camera: **8MP+**
  - CPU: **ARM 64-bit architecture**
  - Storage: **500MB available**

---

## 5. Constraints & Limitations

- Real-time detection may vary in low-light conditions.
- Processing speed depends on the device hardware.
- Only supports **Indian currency notes** in the current version.

---

## 6. Future Enhancements

- Support for multiple currency types.
- Cloud-based fraud detection API integration.
- Improved deep learning models for accuracy enhancement.

---

## 7. References


- OpenCV Documentation
- Android CameraX Guide
- TensorFlow Lite

# Credit Score Calculator Module (Financial Analysis Module)

---

## 1. Introduction

### *1.1 Purpose*

The Credit Score Calculator Module is an essential part of the financial analysis feature in the existing Android application. It enables users to enter financial details and calculates an estimated credit score based on predefined criteria. The module provides a user-friendly UI, performs real-time computations, and presents the results instantly.

### *1.2 Scope*

This module will:

- Be triggered by a button within the main application.
- Allow users to input financial details such as credit utilization, loan history, and inquiries.
- Compute an estimated credit score using predefined weightage factors.
- Display the computed credit score dynamically on the UI.
- Optionally store results in a local database for future reference.

### *1.3 Overview*

This section outlines the functional and non-functional requirements, software dependencies, and constraints for the Credit Score Calculator Module.

---

## 2. Functional Requirements

### *2.1 User Input Collection*

- The module shall collect the following user inputs:
    - Total Credit Limit
    - Credit Used
    - Age of Credit (Months)
    - Number of Loan Accounts
    - Number of Hard Inquiries
    - Payment History (Good, Late, Missed)

### *2.2 Credit Score Calculation*

- The system shall calculate the credit score based on the following weightage:
    - **Payment History**: 35%
    - **Credit Utilization**: 30%
    - **Credit Age**: 15%
    - **Credit Mix (Loan Accounts)**: 10%
    - **Hard Inquiries**: 10%

### *2.3 Score Display*

- The app shall display the computed credit score on the screen dynamically.
- The UI shall provide real-time updates based on user inputs.

### *2.4 Database Storage (Optional)*

- The app shall allow optional storage of credit score history in SQLite or Firebase.
- Users shall be able to retrieve past scores for reference.

### *2.5 User Feedback & Logging*

- The app shall provide suggestions for improving credit scores based on user inputs.
- The app shall maintain a log of score calculations if database storage is enabled.

---

## 3. Non-Functional Requirements

### *3.1 Performance*

- Credit score computation shall take no more than 1 second.
- The module shall work efficiently on devices with 2GB RAM and above.

### *3.2 Compatibility*

- The module shall be compatible with Android 8.0 (API 26) and above.
- It must work on various Android screen sizes and orientations.

### *3.3 Security & Privacy*

- User data shall be processed locally without exposure to external servers.
- If Firebase is enabled, user consent shall be required before data storage.

### *3.4 Scalability*

- The module should allow future enhancements such as credit report visualization and third-party API integration.
- Support for multi-source financial data inputs in future versions.

---

## 4. Software & Hardware Requirements

### *4.1 Software Requirements*

- Android Studio Ladybug 2024.2.2
- Kotlin DSL for build configurations

- Gradle 8.8 or higher
- SQLite for local storage (optional)
- Firebase Firestore (optional for cloud storage)

### *4.2 Hardware Requirements*

- **Minimum Device Specs:**
  - **RAM**: 2GB+
  - **Storage**: 200MB available
  - **CPU**: ARM 64-bit architecture

---

## 5. Constraints & Limitations

- The accuracy of the estimated credit score depends on user-provided data.
- The module does not provide an official credit score but an estimated value.
- Future API integration may require an internet connection.

---

## 6. Future Enhancements

- Credit report PDF export functionality.
- Integration with third-party credit score APIs for accuracy enhancement.
- Improved UI with graphical score analysis and recommendations.

---

## 7. References


- Android SQLite Documentation
- Firebase Firestore Guide
- Financial Credit Score Calculation Standards


---